

Praca z plikami i tekstem

Paweł Gliwny

Po co osobny temat o „tylko” plikach?

W pracy operacje na plikach to codzienność:

- katalogi z setkami obserwacji, zrzutów kamer, plików CSV/JSON/HDF5,
- struktura zagnieżdżona po datach, instrumentach, sesjach,
- potrzeba selekcji po wzorcu (`*_calibrated.h5`),
- konieczność przenośności między Linuxem (klaster), macOS (laptop) i Windowsem (pracownia studencka).

Python oferuje kilka warstw narzędzi, które **częściowo się pokrywają**.

Sztuka polega na *świadomym wyborze*, nie używaniu wszystkiego naraz.

Porównanie narzędzi

Narzędzie	Paradygmat	Mocna strona
<code>os.path + os.listdir</code>	proceduralne, stringi	dziedzictwo, nadal w wielu kodach
<code>glob.glob()</code>	wzorce shell-like	szybkie wyszukiwanie po masce
<code>os.walk()</code>	generator drzewa	pełne przejście rekurencyjne
<code>pathlib.Path</code>	obiektowe	nowoczesne, czytelne, przenośne

W nowym kodzie `pathlib` *powinno być domyślnym wyborem.*

os.path i os.listdir (historyczne)

```
import os
```

```
base = "data/magic/2024"
```

```
for name in os.listdir(base):
```

```
    full = os.path.join(base, name)
```

```
    if os.path.isfile(full) and name.endswith(".csv"):
```

```
        print(full)
```

Wady:

- ścieżki jako stringi — łatwo o błąd (/ vs \, brak slash'a na końcu),
- dużo wywołań os.path.join, os.path.basename, os.path.splitext,
- brak rekurencji (listdir patrzy tylko na jeden poziom).

Dziś rekomenduje się odchodzenie od tego podejścia.

glob.glob() — wzorce w stylu shell

Zwraca listę ścieżek pasujących do wzorca.

```
import glob  
  
files = glob.glob("/data/magic/2024/*/M1_*.fits")
```

Obsługuje wzorce uniksowe:

- * — dowolny ciąg,
- ? — pojedynczy znak,
- [abc] — zbiór znaków,

Zwraca *listę stringów*, nieposortowaną (warto dodać `sorted(...)`).

Kiedy glob jest naturalnym wyborem

- mamy konkretny wzorec nazwy (*_dark.fits, LST_run*.h5),
- struktura katalogów jest *płaska albo o znanej głębokości*,
- wada: nie wiemy czy znaleziony obiekt jest plikiem czy katalogiem, ile waży, kiedy był modyfikowany.

```
# Wszystkie kalibracyjne z jednej nocy  
darks = sorted(glob.glob("/data/2024-11-15/*_dark.fits"))
```

```
# Wszystkie HDF5 w drzewie projektu  
all_h5 = sorted(glob.glob("project/**/*.*h5", recursive=True))
```

```
# Pliki z konkretną liczbą znaków w nazwie  
runs = glob.glob("run_????*.dat")
```

os.walk() — pełne przejście drzewa

Generator, który przechodzi rekurencyjnie przez katalogi i w każdym daje krotkę (dirpath, dirnames, filenames).

- daje kontrolę nad obiema listami (pliki i podkatalogi),
- dobry do skanowania dużych drzew katalogów.

```
import os

for dirpath, dirnames, filenames in os.walk("/data/magic"):
    for fname in filenames:
        if fname.endswith(".fits"):
            full = os.path.join(dirpath, fname)
            print(full)
```

Kiedy `os.walk` zamiast `glob`

Wybierz `os.walk` gdy:

- struktura jest *głęboka i nieregularna*,
- chcesz *decyzji per katalog* (np. czytać tylko podfoldery spełniające warunek),
- musisz pomijać gałęzie (cache, pliki tymczasowe),
- potrzebujesz *logiki podczas przechodzenia*.

Przykład: skanowanie archiwum obserwacji, które mają plik `quality_ok`:

```
for dirpath, dirnames, filenames in os.walk(archive):
    if "quality_ok" not in filenames:
        dirnames[:] = []    # nie schodź głębiej
        continue
    process_night(dirpath, filenames)
```


pathlib.Path — obiektowe podejście

Dostępne od Python 3.4, dziś *zalecane* podejście.

Zalety:

- ścieżki są *obiektami*, nie stringami,
- operator / zamiast `os.path.join`:
`Path("data") / "raw" / "x.fits"`
- *przenośność* — Path automatycznie obsługuje separatory systemowe,
- *metody* zamiast funkcji rozsypanych po `os` i `os.path`.

```
from pathlib import Path
```

```
p = Path("/data/magic/2024/  
M1_run123.fits")
```

```
p.name          # 'M1_run123.fits'  
p.stem          # 'M1_run123'  
p.suffix        # '.fits'  
p.parent        # Path('/data/magic/2024')  
p.parts         # ('/', 'data', 'magic',  
                #  '2024',  
                #  'M1_run123.fits')  
  
p.exists()  
p.is_file()  
p.stat().st_size
```

pathlib: glob i rglob

pathlib ma własne wbudowane wyszukiwanie wzorców.

Co dostajemy:

- *generator* obiektów Path, nie listę stringów,
- każdy element ma od razu pełen interfejs Path
(.stat(), .is_file(), .read_text()),
- rekurencja przez rglob() bez magicznego argumentu recursive=True (zastępuje większość użyć glob.glob(..., recursive=True)).

```
from pathlib import Path

base = Path("/data/magic/2024")

# Tylko jeden poziom
list(base.glob("*.fits"))

# Rekurencyjnie (jak glob z **)
list(base.rglob("*.fits"))

# Bardziej złożone
list(base.glob("M1_*/calibrated/
*.h5"))
```

Wyrażenia regularne (*regex*)

- Wyrażenia regularne to uniwersalny sposób *wyszukiwania i dopasowywania* wzorców w tekście.
- Pojedynczy wzorzec nazywamy *wyrażeniem regex* (ang. *regex pattern*).
- Wzorzec to łańcuch utworzony zgodnie ze *składnią języka wyrażen regularnych*.
- Wbudowany moduł Pythona `re` zapewnia obsługę regexów w kontekście łańcuchów.

Moduł re

Funkcje modułu re można podzielić na trzy kategorie:

- *dopasowywanie* ciągu znaków (`re.search`, `re.match`, `re.findall`),
- *zastępowanie* ciągu znaków (`re.sub`),
- *dzielenie* ciągu znaków (`re.split`).

Wszystkie te operacje są ze sobą związane — wyrażenie regularne *opisuje wzorzec*, który ma zostać znaleziony w tekście, a następnie używamy go do różnych operacji.

Po co nam regex w obróbce danych?

W codziennej pracy z danymi tekstowymi:

- *walidacja*: czy string ma format daty YYYY-MM-DD?
- *ekstrakcja*: wyciągnij datę z nazwy pliku temp_ST01_polesie_2025-03-01.csv,
- *zamiana*: usuń jednostki z kolumny ("18.5 C" → "18.5"),
- *dzielenie*: rozbij log po nieregularnych separatorach.

Alternatywy:

- `str.startswith`, `str.endswith`, `in` — proste, ale nie złapią wzorca,
- `glob` — tylko `*`, `?`, klasy znaków, brak grup i kwantyfikatorów,
- `regex` — *pełna kontrola*, ale wymaga znajomości składni.

Zasada: jeśli `str.startswith` wystarczy, nie sięgaj po `regex`. Jeśli musisz *opisać wzorzec* — `regex`.

Pierwszy przykład

```
import re

nazwa = "temp_ST01_polesie_2025-03-01.csv"
wzorzec = r"\d{4}-\d{2}-\d{2}"

m = re.search(wzorzec, nazwa)
print(m.group())          # '2025-03-01'
print(m.start(), m.end()) # 18 28
```

Co się stało:

- `\d{4}-\d{2}-\d{2}` — wzorzec opisujący datę,
- `r"..."` — *raw string*, bez interpretacji `\` przez Pythona,
- `re.search` szuka pierwszego dopasowania *gdziekolwiek* w tekście,
- `m` to obiekt `Match` — `None` jeśli nic nie znaleziono.

Dlaczego r"..."?

Python interpretuje \ w stringach jako *znak ucieczki*:

```
"\n"      # nowa linia (1 znak)
"\d"     # ostrzeżenie, ale działa (2 znaki: \ i d)
"\\d"    # 2 znaki: \ i d – to widzi regex
```

Bez r musielibyśmy podwajać każdy \ w regexie. *Raw string* wyłącza interpretację:

```
r"\d{4}"  # to samo co "\\d{4}", ale czytelniej
```

Konwencja

Wzorce regex w Pythonie **zawsze** piszemy z prefiksem r.

Metaznaki — klasy znaków

Wzorzec	Znaczenie	Przykład
.	dowolny znak (oprócz nowej linii)	a.c → abc, a3c
\d	cyfra [0-9]	\d\d → 42
\D	<i>nie</i> cyfra	\D+ → abc
\w	znak słowny [A-Za-z0-9_]	\w+ → ST01
\W	<i>nie</i> znak słowny	\W → -,
\s	biały znak (spacja, tab, newline)	\s+ →
\S	<i>nie</i> biały znak	
[abc]	jeden ze znaków a, b lub c	[aeiou]
[^abc]	wszystko <i>oprócz</i> a, b, c	[^0-9]
[a-z]	zakres	[A-Z]+

Metaznaki — kwantyfikatory i kotwice

Wzorzec	Znaczenie	Przykład
*	zero lub więcej	<code>a*</code> → "", a, aaa
+	<i>jeden</i> lub więcej	<code>a+</code> → a, aaa
?	zero lub jeden (opcjonalny)	<code>colou?r</code> → color, colour
{n}	dokładnie n	<code>\d{4}</code> → 2025
{n,m}	od n do m	<code>\d{2,4}</code> → 25, 2025
{n,}	co najmniej n	<code>a{3,}</code> → aaa, aaaa
^	początek stringa	<code>^temp</code>
\$	koniec stringa	<code>\.csv\$</code>
	alternatywa (lub)	<code>csv\ txt</code>

Cztery główne funkcje re

Funkcja	Co robi
<code>re.search(p, s)</code>	szuka <i>gdziekolwiek</i> w s, zwraca pierwszy Match lub None
<code>re.match(p, s)</code>	szuka tylko <i>od początku</i> s (jak ^ na sztywno)
<code>re.findall(p, s)</code>	zwraca <i>listę</i> wszystkich dopasowań
<code>re.finditer(p, s)</code>	generator obiektów Match (dostęp do pozycji, grup)
<code>re.sub(p, r, s)</code>	zamienia dopasowania na r
<code>re.split(p, s)</code>	dzieli s po wzorcu

Jeśli wzorzec *wielokrotnie* używamy, warto go *skompilować*:

```
wzorzec = re.compile(r"\d{4}-\d{2}-\d{2}")  
wzorzec.search(nazwa)    # szybsze w pętli
```

Grupy (. . .)

Nawiasy okrągłe tworzą *grupe* — fragment dopasowania, który można *wyciągnąć*.

```
nazwa = "temp_ST01_polesie_2025-03-01.csv"
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", nazwa)
```

```
m.group(0)    # '2025-03-01'  — całe dopasowanie
m.group(1)    # '2025'       — pierwsza grupa
m.group(2)    # '03'
m.group(3)    # '01'
m.groups()    # ('2025', '03', '01')
```

Grupy numerujemy *od lewej*, licząc otwierające nawiasy. `group(0)` to zawsze całość.

Grupy nazwane — (?P<nazwa>...)

Numerowanie grup szybko robi się *nieczytelne*. Lepiej je nazwać:

```
worzec = r"(?P<rok>\d{4})-(?P<miesiac>\d{2})-(?P<dzien>\d{2})"
m = re.search(worzec, "temp_ST01_polesie_2025-03-01.csv")

m.group("rok")      # '2025'
m.group("dzien")     # '01'
m.groupdict()       # {'rok': '2025', 'miesiac': '03', 'dzien': '01'}
```

Najważniejsza technika tego wykładu

`m.groupdict()` zwraca słownik gotowy do wrzucenia do *DataFrame*. To będzie pomost między regex a pandas.

Grupy nie-przechwytyjące — (?:...)

Czasem nawiasy są nam potrzebne tylko do *grupowania* (np. dla ? lub |), ale *nie chcemy* wyciągać tej grupy.

```
# Z grupą przechwytyjącą:
```

```
re.search(r"(csv|txt)", "plik.csv").groups()    # ('csv',)
```

```
# Z grupą nie-przechwytyjącą:
```

```
re.search(r"(?:csv|txt)", "plik.csv").groups()  # ()
```

Praktyczne zastosowanie — *opcjonalny fragment* wzorca:

```
# Wersja v2 jest opcjonalna
```

```
r"temp_(?P<stacja>\w+)_\d{4}-\d{2}-\d{2}(?:_v(?P<wersja>\d+))?.csv"
```

```
#
```

└ ta grupa istnieje TYLKO

```
#
```

żeby `?` mogło ją złapać

Rozbiór wzorca — pliki temperatury

Wczytujemy pliki o nazwach:

- temp_ST01_polesie_2025-03-01.csv
- temp_ST02_widzew_2025-03-05_v2.csv ← opcjonalna wersja

```
wzorzec = re.compile(  
    r"temp_"  
    r"(?P<stacja>ST\d{2}_\w+?)_"  
    r"(?P<data>\d{4}-\d{2}-\d{2})"  
    r"(?:_v(?P<wersja>\d+))?"  
    r"\.csv"  
)
```

Rozbiór *kawałek po kawałku* na następnym slajdzie.

Rozbiór wzorca — kawałek po kawałku

Fragment	Co robi
temp_	literalny prefiks
(?P<stacja>ST\d{2}_\w+?)	ST + 2 cyfry + _ + nazwa <i>leniwa</i> (+?)
_	separator
(?P<data>\d{4}-\d{2}-\d{2})	data w formacie ISO
(?:_v(?P<wersja>\d+))?	<i>opcjonalna</i> wersja, (?:...) nie- przechwytyjące
\.csv	kropka literalna (\.) + rozszerzenie

Iteracja po plikach z dopasowaniem

```
wzorzec = re.compile(
    r"temp_(?P<stacja>ST\d{2}_\w+?)_"
    r"(?P<data>\d{4}-\d{2}-\d{2})"
    r"(?:_v(?P<wersja>\d+))?\..csv"
)
for plik in Path("dane_lodz/temperatura").glob("*.csv"):
    m = wzorzec.match(plik.name)
    if not m:
        print(f"Pomijam: {plik.name}")
        continue
    meta = m.groupdict()
    print(meta)
# {'stacja': 'ST01_polesie', 'data': '2025-03-01', 'wersja': None}
# {'stacja': 'ST02_widzew', 'data': '2025-03-05', 'wersja': '2'}
```

Zamiana dopasowań na inny tekst — `re.sub`

```
tekst = "Temperatura: 18.5 C, wilgotność: 65 %"
```

```
# Usuń jednostki
```

```
re.sub(r"\s*[CK%]\b", "", tekst)
```

```
# 'Temperatura: 18.5, wilgotność: 65'
```

```
# Zamień format daty: 01.03.2025 → 2025-03-01
```

```
re.sub(
```

```
    r"(\d{2})\.(\d{2})\.(\d{4})",
```

```
    r"\3-\2-\1",
```

```
    "01.03.2025"
```

```
)
```

```
# '2025-03-01'
```

`\1`, `\2`, `\3` w stringu zastępującym to *backreferences* do grup. Z grupami nazwanymi: `\g<nazwa>`.

Flagi

Modyfikują zachowanie wzorca:

Flaga	Działanie	Skrót
re.IGNORECASE	bez rozróżniania wielkości liter	re.I
re.MULTILINE	^ i \$ działają per linia	re.M
re.DOTALL	. łapie też nową linię	re.S
re.VERBOSE	pozwała na białe znaki i komentarze w regex	re.X

```
re.search(r"csv", "plik.CSV", flags=re.IGNORECASE) # Match: 'CSV'
re.compile(r"""
    temp_                # prefiks
    (?P<stacja>ST\d{2}_\w+?)_ # stacja (leniwe)
    (?P<data>\d{4}-\d{2}-\d{2}) # data ISO
    \.csv
""", flags=re.VERBOSE)
```

Typowe pułapki

- **Zachłanne kwantyfikatory** — `.+` zżera za dużo. Używaj `.+?` lub *negowanych klas* `([^\_]+)`.
- **Niedopasowanie końca** — `re.match` $\neq$ `re.fullmatch`. `match` dopasowuje od początku, ale *nie wymaga* pasowania do końca.
- **Kropki w rozszerzeniach** — `.csv` w regex to „dowolny znak + csv”. Pisz `\.csv`.
- **Polskie znaki** — `[a-z]` ich nie złapie. Użyj `\w` (z `re.UNICODE`, domyślne w Py3) albo jawnego zakresu.
- **Niezamierzona alternatywa** — `cat|dog files` znaczy "cat" *lub* "dog files". Grupuj: `(?:cat|dog) files`.

- **Niezekapowane znaki specjalne** — (,), +, *, ?, ., |, [,] mają znaczenie.

Literalnie: poprzedź \.

Narzędzie do debugowania: **regex101.com** — pokazuje co dopasowało się znak po znaku, wraz z wyjaśnieniem składni.

Podsumowanie regex

Do zapamiętania:

- regex opisuje *wzorzec*, nie konkretny string,
- w Pythonie zawsze `r"..."` (raw string),
- cztery podstawowe operacje: `search`, `findall`, `sub`, `split`,
- *grupy nazwane* (`?P<nazwa>...`) + `m.groupdict()` to most do pandas,
- kompiluj wzorce używane wielokrotnie (`re.compile`).

Na ćwiczeniach:

- 3 różne wzorce dla 3 typów plików (temperatura, PM2.5, wilgotność),
- każdy z grupami nazwanymi (stacja, data, opcjonalnie wersja),
- wynik `m.groupdict()` dołożymy jako kolumny w `DataFrame`.

Od regex do pandas — kluczowa intuicja

Wynik `m.groupdict()` to *słownik* — a lista słowników to gotowy DataFrame:

```
import pandas as pd

rekordy = [
    {"stacja": "ST01_polesie", "data": "2025-03-01", "wersja": None},
    {"stacja": "ST02_widzew", "data": "2025-03-05", "wersja": "2"},
    {"stacja": "ST03_baluty", "data": "2025-03-02", "wersja": None},
]
pd.DataFrame(rekordy)
```

Wzorzec, który zobaczymy wiele razy

pathlib daje pliki → `regex.match(plik.name).groupdict()` daje metadane → lista słowników → `pd.DataFrame` lub *kolumny dodawane do wczytanego CSV*.

Wczytanie jednego pliku — proste

Mamy plik temp_ST01_polesie_2025-03-01.csv:

```
plik = Path("dane_lodz/temperatura/temp_ST01_polesie_2025-03-01.csv")
df = pd.read_csv(plik)
df.head()
#           timestamp  temperatura_C uwagi
# 0  2025-03-01 00:00:00           16.53   ok
# 1  2025-03-01 03:00:00            7.03  NaN
# ...
```

Problem: w środku CSV-a *nie ma kolumny stacja*. Informacja o stacji jest tylko w *nazwie pliku*.

Dlatego musimy ją wyciągnąć regexem i *dokleić do ramki*.

Dokładanie metadanych do ramki

```
wzorzec = re.compile(
    r"temp_(?P<stacja>ST\d{2}_\w+?)_"
    r"(?P<data>\d{4}-\d{2}-\d{2})"
    r"(?:_v(?P<wersja>\d+))?.csv"
)

m = wzorzec.match(plik.name)
meta = m.groupdict()          # {'stacja': 'ST01_polesie', ...}

df = pd.read_csv(plik)
df["stacja"] = meta["stacja"] # broadcast – wartość trafia do każdego wiersza
df["data_pliku"] = meta["data"]
df["wersja"] = meta["wersja"] or "1"
```

Przypisanie skłara do kolumny w DataFrame *powtarza wartość* dla każdego wiersza — to się nazywa *broadcasting*.

Pełna pętla — pathlib + regex + pandas

```
def wczytaj_temperature(root: Path) -> pd.DataFrame:
    ramki = []
    for plik in root.rglob("temp_*.csv"):
        m = wzorzec.match(plik.name)
        if not m:
            print(f"Pomijam: {plik.name}")
            continue
        meta = m.groupdict()
        df = pd.read_csv(plik)
        df["stacja"] = meta["stacja"]
        ...
        ramki.append(df)
    return pd.concat(ramki, ignore_index=True)

temp = wczytaj_temperature(Path("dane_lodz"))
print(temp.shape)    # (320, 7)
```

Idiom: lista ramek + `pd.concat`

Najczęstszy schemat przy łączeniu wielu plików:

```
ramki = []                # 1. pusta lista
for plik in pliki:
    df = pd.read_csv(plik)
    # ...przetwarzanie, dodanie kolumn...
    ramki.append(df)      # 2. dorzucamy do listy
wynik = pd.concat(ramki) # 3. jedno wywołanie na końcu
```

Antywzorzec — czego NIE robić

```
wynik = pd.DataFrame()
for plik in pliki:
    df = pd.read_csv(plik)
    wynik = pd.concat([wynik, df]) # ❌  $O(n^2)$ !
```

Każdy concat w pętli kopiuje *całą dotychczasową ramkę*. Przy 100 plikach to katastrofa. Buduj *listę*, łącz *raz*.

Czym jest `pd.concat`?

Konkatenacja to „sklejenie” obiektów wzdłuż wybranej osi. Najprostszy obraz:

Wzdłuż wierszy (`axis=0`, domyślnie):

- ramki muszą mieć *te same kolumny* — wyniki układają się *jeden pod drugim*.
- Używamy gdy: każdy plik to ten sam typ pomiaru, tylko inna stacja/data.

Wzdłuż kolumn (`axis=1`):

- ramki muszą mieć *ten sam indeks* — wyniki układają się *obok siebie*.
- Używamy gdy: różne pomiary tych samych obiektów (rzadziej w obróbce CSV).

```
pd.concat([df1, df2, df3])           # wiersze
pd.concat([df1, df2], axis=1)        # kolumny
pd.concat([df1, df2], ignore_index=True) # przenumeryj indeks 0..N
```

concat to nie merge!

Częsta pomyłka u początkujących:

	pd.concat	df.merge / pd.merge
Cel	<i>sklej ramki wzdłuż osi</i>	<i>połącz po wspólnej kolumnie</i>
Analogia SQL	UNION ALL	JOIN
Wymaga	zgodnych kolumn (lub indeksów)	wspólnego klucza
Wynik	więcej wierszy lub więcej kolumn	wiersze <i>dopasowane</i> po kluczu

Praktycznie:

- *wczytujemy 30 plików temperatury* → concat (wszystkie mają te same kolumny),
- *dokładamy PM2.5 do temperatury po stacji i czasie* → merge (różne kolumny, wspólny klucz).

Oba spotkamy w naszym pipeline.

ignore_index — drobiazg, który ratuje życie

Bez tego argumentu indeksy z każdej ramki *zostają zachowane*:

```
df1 = pd.DataFrame({"x": [1, 2]}) # indeks 0, 1
```

```
df2 = pd.DataFrame({"x": [3, 4]}) # indeks 0, 1
```

```
pd.concat([df1, df2])
```

```
#      x
```

```
# 0    1
```

```
# 1    2
```

```
# 0    3 ← duplikat indeksu!
```

```
# 1    4
```

```
pd.concat([df1, df2], ignore_index=True)
```

```
#      x
```

```
# 0    1
```

```
# 1    2
```

```
# 2    3 ← ciągła numeracja
```

```
# 3    4
```


concat z keys — MultiIndex jako wartość dodana

Czasem warto *zachować informację* skąd pochodzi dany kawałek:

```
kawalki = {}  
for plik in Path("dane_lodz/temperatura").glob("temp_*.csv"):  
    kawalki[plik.stem] = pd.read_csv(plik)
```

```
duza = pd.concat(kawalki, names=["plik_zrodlowy", "wiersz"])  
duza.head()
```

```
#                                     timestamp  temperatura_C  
# plik_zrodlowy      wiersz  
# temp_ST01_polesie_2025-03-01 0      2025-03-01 00:00:00      16.53  
#                               1      2025-03-01 03:00:00       7.03  
# temp_ST01_polesie_2025-03-02 0      2025-03-02 00:00:00      15.21
```

Alternatywa do *dodawania kolumny ze źródłem* — różnica stylistyczna, ale MultiIndex pozwala potem łatwo robić `groupby(level=0)`.

Walidacja — co jeśli plik nie pasuje?

W realnych danych zawsze jest *coś*, co nie pasuje (README, backups, _old).

Strategia: regex jako *filtr* + jawne logowanie.

```
ramki = []
pominieta = []
for plik in root.rglob("*.csv"):
    m = wzorzec.match(plik.name)
    if not m:
        pominieta.append(plik.name)
        continue
    df = pd.read_csv(plik)
    df = df.assign(**m.groupdict())    # ← elegancki sposób
    ramki.append(df)
print(f"Wczytano: {len(ramki)}, pominięto: {len(pominieta)}")
if pominieta:
    print("Sprawdź:", pominieta[:5])
```

Podsumowanie — pathlib + regex + pandas

Wzorzec, który właśnie poznaliśmy:

1. **Znajdź pliki** — `Path(root).rglob("*.csv")`
2. **Dopasuj wzorzec** — `re.compile(...)` z grupami nazwanymi
3. **Pomiń niepasujące** — `if not m: continue`
4. **Wczytaj** — `pd.read_csv(plik)`
5. **Wzbogac** — kolumny z `m.groupdict()` (broadcasting lub `df.assign(**...)`)
6. **Zbierz** — `ramki.append(df)` w pętli
7. **Sklej raz** — `pd.concat(ramki, ignore_index=True)` po pętli

Wynik: *jedna ramka* ze wszystkich plików, z metadanymi w kolumnach. Gotowa do groupby, merge, filtrowania, wizualizacji.